

武汉理工大学

《企业级应用软件设计与开发》

课程大作业

基于 Agentic AI 与 MCP 的 小桃旅游助手

姓名：马锦涛

学号：2025302947

班级：研 255

专业：计算机技术

指导教师：戚欣

2026 年 6 月 19 日

目录

- 摘要 4
- 一、选题背景与设计思想 5
 - 1.1 问题定义 5
 - 1.2 现有方案不足 5
 - 1.3 项目价值 5
 - 1.4 技术路线 6
- 二、Specs 规格文档..... 7
 - 2.1 Product Spec 7
 - 2.2 Architecture Spec 7
 - 2.3 API Spec 8
 - 2.4 Tool Spec 8
- 三、系统架构与设计 9
 - 3.1 核心架构图 9
 - 3.2 Agent 交互流程 9
 - 3.3 RAG 数据流设计 9
 - 3.4 Docker 部署架构 10
- 四、关键实现与代码展示 11
 - 4.1 Agent 核心循环 11
 - 4.2 工具定义与 MCP 配置 11
 - 4.3 SSE 流式状态返回 11
 - 4.4 前端交互与 AI 辅助开发 12

五、测试与评估	14
5.1 功能测试矩阵	14
5.2 Agent 行为评估	14
5.3 Demo 展示与部署验证	14
5.4 异常兜底验证	15
六、系统升级与扩展	16
6.1 可扩展架构	16
6.2 下一阶段计划	16
6.3 AI 能力演进路径	16
七、课程总结	17
7.1 项目完成情况	17
7.2 个人收获	17
7.3 工程思维转变	17
7.4 对课程的建议	17
参考资料	18

摘要

本课程报告围绕《企业级应用软件设计与开发》中 Agentic AI 原生开发方向的要求，完成了一套面向旅行规划场景的智能体系统：小桃旅游助手。系统以自然语言旅行需求为入口，结合 DashScope/Qwen、DeepSeek R1、LangChain ReAct Agent、MCP 工具协议和 ChromaDB 向量数据库，实现从需求理解、工具调用、攻略检索到行程生成的完整闭环。用户可以输入出发地、目的地、日期、预算和同行人员偏好，系统自动查询交通、天气、住宿、黄历、航班等信息，并通过 SSE 将 Agent 执行状态实时展示在前端。项目同时完成了 Docker 化、本地运行和云服务器部署，形成了可访问、可演示、可扩展的课程工程交付物。

关键词

企业级应用； Agentic AI； MCP； LangChain； ReAct Agent； RAG； FastAPI； Docker； 旅行规划

一、选题背景与设计思想

1.1 问题定义

本项目聚焦个人与家庭旅行规划中的多源信息整合问题。一次完整旅行通常需要同时考虑交通路线、火车票或航班、天气变化、酒店住宿、预算约束、同行人的身体状况、当地攻略和文化习俗。传统方式下，用户往往需要在地图、订票、天气、酒店、攻略社区等多个应用之间反复切换，并手动汇总信息，规划过程耗时且容易遗漏关键约束。

因此，本项目将问题定义为：构建一个能够理解自然语言旅行需求、自动调用外部工具、结合本地攻略知识并生成完整旅行方案的 Agentic AI 原生应用。系统目标不是单纯聊天，而是让智能体真正完成需求拆解、工具选择、结果整合和方案输出。

1.2 现有方案不足

现有旅行规划方案大致分为搜索引擎、攻略社区、OTA 平台和通用大模型问答四类。搜索引擎覆盖面广但信息分散；攻略社区经验丰富但难以结合实时天气和票务；OTA 平台擅长预订但缺少跨工具综合规划；通用大模型回答流畅但容易缺乏实时数据支撑。对于课程要求中的企业级应用设计而言，有价值的系统应当能够把大模型推理、外部工具和本地知识库组合起来，形成可追踪、可部署、可扩展的工程闭环。

1.3 项目价值

维度	说明
用户价值	将交通、天气、住宿、攻略和预算整合到一次对话中，降低用户查询和比对成本。
技术价值	展示如何用 LangChain ReAct Agent 编排 MCP 工具、RAG 检索和大模型推理。
工程价值	形成前后端分离、SSE 流式状态、Docker 部署和云服务器访问的完整交付链路。
课程价值	覆盖 SDD 规格设计、Agent 架构、工具调用、测试评估和升级扩展等评分点。

1.4 技术路线

系统采用“前端交互 - API 网关 - Agent 编排 - MCP 工具 - RAG 知识库 - 云端部署”的技术路线。前端使用 Vue 3 构建旅行任务舱界面，后端使用 FastAPI 承接 API 与 SSE 流式响应，Agent 层使用 LangChain ReAct 模式完成多步骤推理，工具层通过 MCP 接入 12306、高德、航班、黄历和搜索服务，知识层通过 ChromaDB 保存用户上传攻略文档，部署层使用 Docker Compose 和 Nginx 将前端与后端组合到云服务器。

层次	技术选型	作用
前端	Vue 3 + Vite + TypeScript + Pinia	聊天、状态时间线、文档上传、工具状态展示
后端	FastAPI + Pydantic + Uvicorn	API 网关、SSE 流、文件上传、健康检查
Agent	LangChain ReAct Agent	需求预分析、工具选择、多步骤推理
LLM	DashScope/Qwen + DeepSeek R1	通用理解生成与复杂需求深度分析
工具	MCP + LangChain Tool	交通、天气、住宿、航班、黄历等外部能力
RAG	ChromaDB + Retriever	攻略文档向量化与知识检索
部署	Docker + Nginx + 云服务器	本地/云端统一运行环境

二、Specs 规格文档

2.1 Product Spec

产品规格从目标用户、核心场景、功能优先级和边界约束四个角度定义系统能力，确保实现过程不是“为了使用 AI 而使用 AI”，而是围绕旅行规划的真实痛点构建智能体。

项目	说明
目标用户	需要快速制定旅游方案的个人、学生、家庭用户，尤其适合预算、天气、父母同行等复杂约束场景。
核心场景	用户输入出发地、目的地、时间、预算和偏好后，系统生成交通、住宿、天气、行程、注意事项和预算建议。
核心价值	通过 Agent 自动调用工具并整合结果，让用户从“自己查很多 App”变为“提出需求后获得综合方案”。
边界约束	系统提供规划建议，不直接完成购票、支付和酒店预订；外部 MCP 服务异常时给出可读兜底。

模块	功能	优先级
对话规划	自然语言输入旅行需求，流式返回规划方案	P0
工具调用	自动查询火车票、天气、住宿、航班、黄历、路线	P0
RAG 文档	上传攻略文档并作为本地知识源检索	P0
状态展示	展示 Agent 正在分析、R1 推理、工具查询和完成状态	P1
部署交付	Docker Compose 本地运行与云服务器公网访问	P1
可观测性	后续接入 Tracing、Benchmark 和调用链日志	P2

2.2 Architecture Spec

层次	模块	职责
前端层	TravelDesk、ChatComposer、ControlPanel、StatusTimeline	提供聊天输入、消息渲染、状态时间线、文档上传与工具状态展示。
接口层	FastAPI routers: chat、documents、tools	提供聊天、流式响应、文档上传、工具状态和健康检查接口。
业务层	AgentService、SSE service、schemas	组织需求预分析、Agent 执行、工具包装、异常兜底和结果返回。
AI 层	DashScope Qwen、DeepSeek R1	完成自然语言理解、复杂推理、方案生成和深度分析。
工具层	MCPToolManager、tool_registry	管理 MCP SSE 连接并把远端工具包装为 LangChain Tool。
知识层	RAG tool、ChromaDB	文档加载、分块、向量化、检索和持久化。

层次	模块	职责
部署层	Dockerfile、docker-compose、Nginx	托管前端静态文件，反向代理 API，支持云端部署。

2.3 API Spec

接口	方法	请求	响应	说明
/api/health	GET	无	status/service/frontend	后端健康检查
/api/tools	GET	无	tools/count/mcp_available/rag_total	工具和 RAG 状态
/api/documents/upload	POST	multipart files	uploaded/files/message	上传攻略文档并构建索引
/api/chat	POST	message/messages/max_iterations	answer/scenario/extraction	同步聊天接口
/api/chat/stream	POST	message/messages/max_iterations	SSE status/analysis/final/error	流式聊天和状态展示

2.4 Tool Spec

本地工具	远端服务	远端工具	用途
train_query	12306 Server	get-tickets	查询火车票并结合自驾路线对比。
gaode_weather	Gaode Server	maps_weather	查询天气。
gaode_hotels_search	Gaode Server	maps_text_search	搜索酒店和民宿。
gaode_poi_search	Gaode Server	maps_text_search	搜索景点、餐饮和城市 POI。
gaode_driving	Gaode Server	maps_direction_driving	查询驾车路线。
flight_query	flight Server	search_flight	查询国内航班价格与时刻表。
lucky_day	bazi Server	getChineseCalendar	查询黄历宜忌。
rag_search	本地 ChromaDB	Retriever	检索用户上传攻略文档。

三、系统架构与设计

3.1 核心架构图

系统整体采用前后端分离与 Agent 编排结合的架构。前端只负责用户交互和状态展示，后端集中处理 Agent 推理和工具调用，外部 MCP 服务提供实时数据能力，RAG 模块提供本地攻略知识增强。

```
用户浏览器
-> Vue 3 前端 (聊天 / 上传 / 状态时间线)
-> FastAPI API (/api + SSE)
-> LangChain ReAct Agent
-> MCP 工具 (12306 / 高德 / 航班 / 黄历 / 搜索)
-> RAG 知识库 (ChromaDB)
-> DashScope Qwen / DeepSeek R1
```

3.2 Agent 交互流程

Agent 的执行过程不是一次性调用模型，而是包含预分析、场景判断、工具选择、Observation 整合和最终生成。复杂需求会触发 R1 深度分析，用于预算、多约束和多目的地路线优化。

```
用户输入需求 -> Qwen 结构化预分析 -> 判断 simple/complex/multi_destination
-> 创建 ReAct Agent -> 调用 R1 / MCP / RAG 工具 -> 汇总 Observation
-> 生成最终方案 -> SSE 推送给前端
```

3.3 RAG 数据流设计

RAG 数据流从用户上传攻略文档开始。系统支持 TXT、MD、PDF、CSV 等格式，将文档加载后切分为 chunk，再通过 DashScope embedding 生成向量并写入 ChromaDB。聊天时，Agent 可以通过 rag_search 获取与目的地、景点、美食、游玩时间相关的本地知识。

```
攻略文档上传 -> 文档解析 -> 文本切分 -> DashScope Embedding -> ChromaDB
用户问题 -> Retriever 检索 Top-K -> 注入 Agent 上下文 -> 生成增强答案
```

3.4 Docker 部署架构

生产部署采用 Docker Compose。前端镜像先构建 Vue 静态资源，再由 Nginx 托管；Nginx 将 /api 请求反向代理到 backend 容器。后端仅绑定服务器本机 127.0.0.1:8000，公网只开放 80 端口，从而减少后端直接暴露风险。

容器	端口	职责
frontend	80:80	Nginx 托管 Vue 静态文件，反向代理 /api。
backend	127.0.0.1:8000:8000	FastAPI、Agent、MCP、RAG 服务。
chromadb_data volume	内部挂载	保存 ChromaDB 向量数据。

四、关键实现与代码展示

4.1 Agent 核心循环

AgentService 是系统核心。它负责加载环境变量、初始化 Qwen 模型、连接 MCP manager、构建 LangChain Tool、执行 ReAct Agent，并通过回调函数把状态写入 SSE 队列。

```
pre analyze query(message)
-> detect_multi_destination()
-> create_tools()
-> create_react_agent(llm, tools, prompt)
-> AgentExecutor(..., max_iterations)
-> stream status / analysis / final
```

这种设计让模型不再只是回答问题，而是能够根据上下文主动选择工具。例如家庭出行、预算约束或多目的地场景会触发 R1 分析；普通城市旅行则优先查询交通、天气、住宿和攻略信息。

4.2 工具定义与 MCP 配置

工具定义由 tool_registry.py 统一维护，包括工具名称、自然语言描述、参数 schema、MCP server 名称和远端工具名。这样做的好处是工具能力对 Agent 可解释，后续新增 MCP server 时也能保持扩展性。

实现点	说明
统一注册表	所有工具在 AVAILABLE_TOOLS 中声明，避免工具散落在业务代码中。
参数归一化	后端将用户输入中的 origin/from/dep 等字段统一转换为远端工具需要的参数。
异常兜底	远端未知工具、黄历超时、12306 异常会转化为用户可读提示。
真实修复案例	航班工具从 searchFlightsByDepArr 修正为远端实际的 search_flight。

4.3 SSE 流式状态返回

为了让用户看到 Agent 的执行过程，后端采用 text/event-stream 返回 status、analysis、final、error 等事件。前端逐段解析 SSE，并把执行状态展示在 Agent 工况时间线中。

```
event: status
data: {"message":"正在分析您的旅行需求..."}
```

```
event: analysis
data: {"scenario_type":"complex", "needs_r1":true}

event: final
data: {"answer":"完整旅行方案..."}
```

4.4 前端交互与 AI 辅助开发

前端界面采用“旅行任务舱”风格设计，包含主对话区、控制面板和 Agent 工况区。控制面板提供攻略文档接入、最大迭代次数调整、工具在线状态和 RAG 文档统计；工况区展示 R1 推理状态、事件缓存和实时执行日志。



图 4.1 控制面板：攻略文档、规划配置与工具状态



图 4.2 Agent 工况：R1 推理、事件缓存与执行时间线

AI IDE 使用方面，本项目在开发过程中借助 AI 辅助完成需求拆解、Docker 部署排错、MCP 工具 schema 对齐、README 与报告整理等任务。

五、测试与评估

5.1 功能测试矩阵

测试项	测试方式	预期结果	结果
本地前端访问	打开http://localhost:8080	Vue 页面正常加载	通过
云服务器访问	打开http://175.178.129.222	公网可访问系统	通过
健康检查	GET /api/health	返回 status=ok	通过
工具状态	GET /api/tools	返回 9 个工具且 MCP 在线	通过
复杂需求	家庭出行、预算、天气约束	触发 R1 深度分析并生成方案	通过
航班工具	武汉到上海指定日期	返回航班列表	通过
文档上传	上传 TXT/MD/PDF/CSV	构建 RAG 索引	已实现

5.2 Agent 行为评估

评估维度	观察指标	说明
工具调用合理性	是否根据需求选择交通、天气、住宿、黄历、航班工具	复杂旅行需求会触发多工具查询。
过程可解释性	前端是否展示状态时间线	用户能看到分析、R1、工具查询和完成状态。
答案完整性	是否包含交通、天气、住宿、行程、预算和注意事项	Demo 输出覆盖主要旅行规划要素。
异常鲁棒性	远端 MCP 异常时是否有可读提示	已对黄历超时、12306 异常、未知工具做友好处理。
部署可复现性	Docker Compose 与云服务器是否可运行	前端 Nginx + 后端 FastAPI 均可容器化部署。

5.3 Demo 展示与部署验证

系统已部署到云服务器，访问地址为 <http://175.178.129.222>。下图展示了用户输入“后天从南京去武汉旅游，和父母一起，为期一周，预算 10000 元”后，系统生成包含交通、天气、住宿和注意事项的综合方案。



图 5.1 云端 Demo：旅行方案生成效果

5.4 异常兜底验证

异常类型	原始风险	系统处理
航班工具名变化	远端返回 Unknown tool	通过列工具确认 schema，将工具名修正为 search_flight。
黄历服务超时	底层 MCP 错误暴露给用户	转换为“黄历查询暂不可用，可先按常规日期规划”。
12306 查询异常	远端 JS 错误影响最终回答	转换为“火车票查询暂不可用，可能日期超范围或服务异常”。
Docker 平台不匹配	Mac arm64 镜像无法在 amd64 云服务器运行	build-images.sh 默认构建 linux/amd64 镜像。

六、系统升级与扩展

6.1 可扩展架构

当前系统已经将前端、后端、Agent、工具和 RAG 分层。后续新增能力时，可以在 `tool_registry` 中注册新工具，在 `servers_config` 中增加 MCP server，并在 `AgentService` 中补充参数归一化，不需要重写核心对话流程。

6.2 下一阶段计划

接入 HTTPS、域名和自动续期证书，提升云端访问体验。

增加 LangSmith 或自建 tracing，记录每次 Agent 的工具调用链路、耗时和失败原因。

构建 20-50 条旅行需求 Benchmark，统计成功率、工具调用覆盖率和平均响应时间。

完善文档管理能力，支持已上传攻略的删除、重新索引和来源引用。

准备 Ollama 本地模型兜底，降低 Demo Day 外部 API 失效风险。

6.3 AI 能力演进路径

系统后续可以从单 Agent 升级为多智能体协作：交通 Agent 专注火车与航班，住宿 Agent 专注酒店和区域选择，天气 Agent 评估出行风险，预算 Agent 负责费用估算，最终由 Planner Agent 汇总多方建议。长期记忆方面，可以保存用户常用出发地、预算偏好、酒店偏好和同行人情况，使下一次规划更个性化。

七、课程总结

7.1 项目完成情况

本项目完成了从需求定义、规格设计、前后端实现、Agent 编排、MCP 工具接入、RAG 知识库、Docker 部署到云服务器访问的完整闭环。与单纯调用大模型不同，小桃旅游助手强调模型、工具、状态和工程部署的组合设计，符合课程关于 Agentic AI 原生开发的实践方向。

7.2 个人收获

通过本项目，我更加明确了企业级 AI 应用不是“给系统加一个聊天框”，而是要将大模型能力嵌入真实业务链路。Agent 的价值来自工具可用性、上下文组织、异常兜底和可观测状态，而不仅是最终回答文本。

7.3 工程思维转变

传统开发关注接口、页面和数据库，Agentic AI 开发还需要关注工具协议、Prompt 规格、模型边界、执行状态、评估样例和部署安全。从“代码编写者”到“智能体编排者”的转变，体现在如何把 LLM、MCP、RAG、前端状态和 Docker 部署编排成一个可靠系统。

7.4 对课程的建议

建议课程后续继续增加 Agent 工程化案例，例如 MCP 工具设计、LangGraph 状态机、LLMOps 追踪和 Benchmark 构建。相比只讲模型调用，这些内容更能帮助学生理解 AI 原生应用与传统软件工程之间的衔接。

参考资料

LangChain 官方文档: <https://python.langchain.com/>

FastAPI 官方文档: <https://fastapi.tiangolo.com/>

Model Context Protocol: <https://modelcontextprotocol.io/>

Docker 官方文档: <https://docs.docker.com/>

DeepSeek 开放平台: <https://platform.deepseek.com/>

阿里云 DashScope 文档: <https://dashscope.aliyun.com/>